

プログラミング入門2

第9回 構造体

篠埜 功

今回の内容

- 構造体

構造体とは

学生の身体検査のデータの型

```
char  name[20];    /* 名前 */  
int    height;     /* 身長 */  
double weight;     /* 体重 */
```

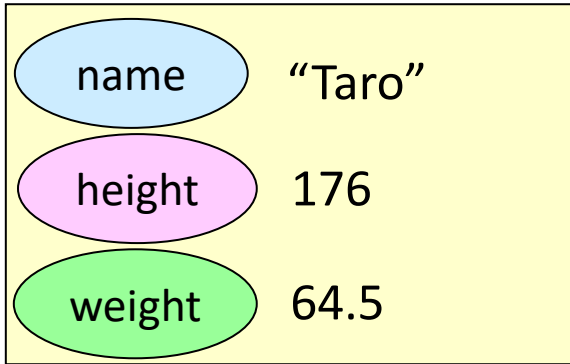
太郎君の身体検査のデータ

```
name:  "Taro"  
height: 176  
weight: 64.5
```

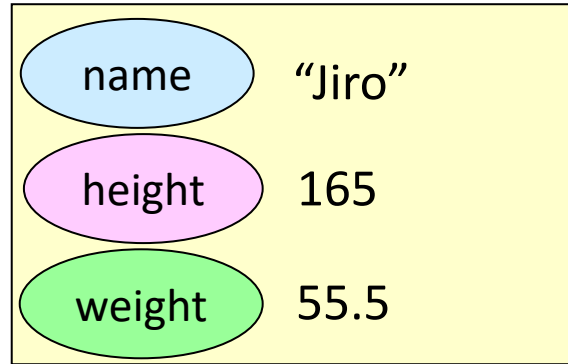
このようなデータを一つのかたまりとして扱いたい。

構造体とは

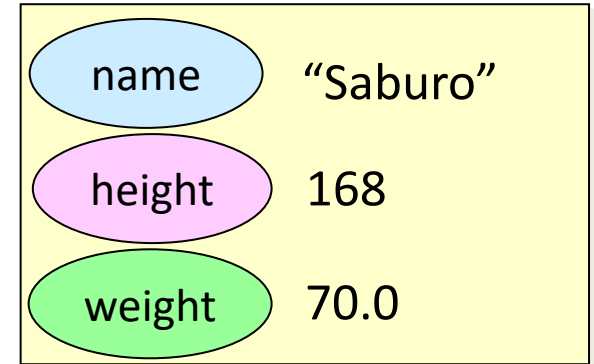
構造体とは、複数の型のデータをひとまとめにしたデータ構造



構造体の例1



構造体の例2

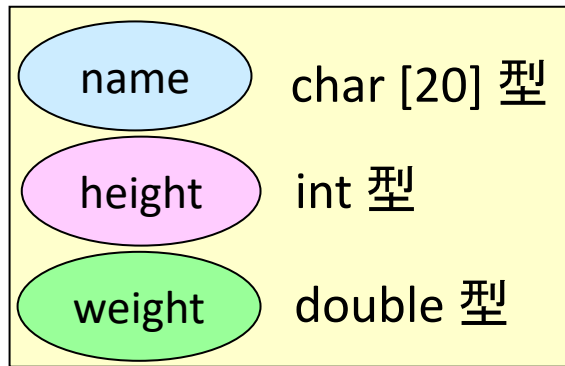


構造体の例3

構造体型

構造体型は、複数の型を組み合わせ得られる型である。

(構造体型の例)



前ページの例1、2、3の構造体の型は上記の構造体型である。

(補足) 構造体型は、型を組み合わせ得られる型である。このようなものを派生型 (derived type) (教科書 p. 342) という。(プログラミング言語一般には複合型 (compound type あるいは composite type) という。C では composite type は複合型ではなく合成型という別の意味を持つ用語である。) 配列型、ポインタ型も複合型である。int 型、double 型、char 型等は基本型である。構造体型を組み合わせ構造体型を作ってもよく、配列型、ポインタ型、共用体型、関数型など自由に組み合わせる良い。

構造体型を表す型式

int, doubleなど、型を表す式を型式(type expression)という。
構造体型を表す式は以下のような形で記述する。

(例)

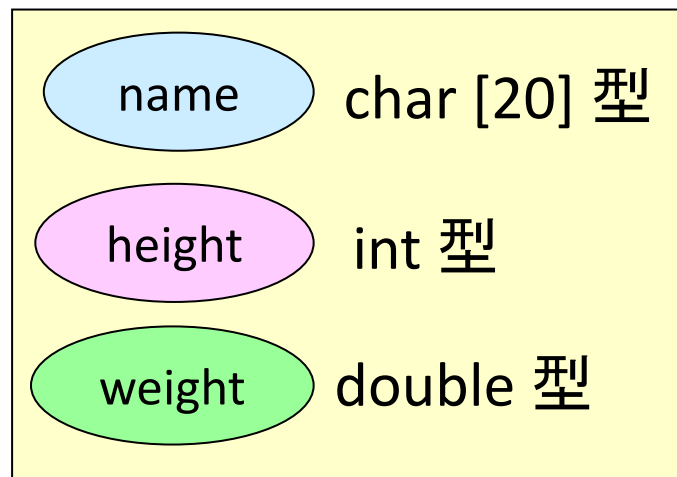
struct {

char name[20];

int height;

double weight;

}



構造体型を表す型式の構文

```
struct {  
    変数宣言  
    変数宣言  
    ...  
}
```

(キーワードstruct の後、中括弧の中に
変数宣言を複数個並べる)

構造体型の変数の宣言

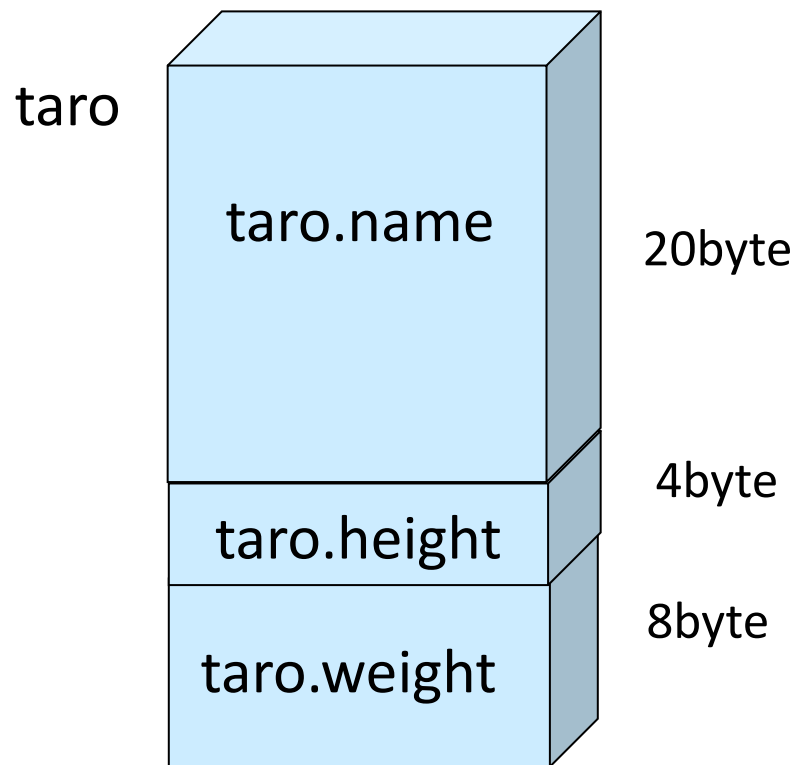
構造体型の変数を宣言できる。int型、double型の変数宣言と同様、型式の後に変数名を書き、セミコロンを書いて宣言する。

τ 変数名; (τは構造体型を表す型式とする)

(例) 赤字の部分は構造体型
 を表す型式

```
struct {  
    char name[20];  
    int height;  
    double weight;  
} taro;
```

↑
青字の部分は宣言
する変数名



(バイト数は演習室の環境の場合)

構造体のメンバー

前ページの例で説明する。

```
struct {  
    char name[20];  
    int height;  
    double weight;  
} taro;
```

name, height, weightを、構造体taroのメンバーという。

構造体のメンバーアクセス

式 e が、名前 m のメンバーを持つ構造体型の式するとき、 $e.m$ で構造体のメンバーが得られる。 $.$ をドット演算子という。

(例) 前ページのようにtaroという変数を宣言すると、
`taro.name`, `taro.height`, `taro.weight`でtaroの各メンバーが得られる。

例（打ち込んで確認）

```
#include <stdio.h>
#include <string.h>
int main (void) {
    struct {
        char name[20];
        int height;
        double weight;
    } taro;
    strcpy (taro.name, "Taro");
    taro.height = 176;
    taro.weight = 64.5;
    printf ("%sの身長は%dcм、体重は%fkgです。¥n",
        &(taro.name[0]),
        taro.height,
        taro.weight);
    return 0;
}
```

文字列を配列に代入するときに
strcpyを用いると便利が良い。

&(taro.name[0])はtaro.name
と書いても同じ意味である。

構造体型の変数の初期化

```
#include <stdio.h>
int main(void)
{
    struct {
        char name[20];
        int height;
        double weight;
    } taro = {"Taro", 176, 64.5};
    printf("%sの身長は%dcm、体重は%fkgです。¥n",
        taro.name, taro.height, taro.weight);
    return 0;
}
```

構造体の初期化は{ }を使って記述する。それぞれ対応するメンバーが初期化される。
char型の配列の初期化は、
char name [20] = "Taro";
のように書いてよい(第8回資料参照)。

(注意) char型の配列nameに対し、name="Taro"のように代入することはできない。(初期化と代入は異なる)

typedefの使用(教科書p. 194)

構造体を使うとき、構造体型に、typedefで名前を付けると便利がよい。

typedefは、

typedef <変数宣言の変数名部分を新しくつける型名でおきかえたもの>の形で使う。

(例1) **typedef** int **aaa**; と宣言すると、aaaはintの別名。

(例2) **typedef** int **bbb** [3]; と宣言すると、bbbはint [3]型の別名。

(例3) **typedef** struct {
 char name[20];
 int height;
 double weight;

} **student**;

と宣言すると、studentは

struct {
 char name[20];
 int height;
 double weight;
} 型の別名。

構造体型に名前をつける例(打ち込んで確認)

```
#include <stdio.h>
int main(void)
{
    typedef struct{
        char  name[20]; /* 名前 */
        int   height;    /* 身長 */
        double weight;   /* 体重 */
    } student;
    student taro = {"Taro", 176, 64.5};
    printf("%sの身長は%dcm、体重は%fkgです。¥n",
           taro.name, taro.height, taro.weight);
    return 0;
}
```

構造体の代入 (教科書 p. 340)

同じ型の構造体であれば、代入することが可能

```
student taro;  
student jiro = {"Jiro", 165, 55.5};  
...  
taro = jiro;
```

taro=jiroの代入式によって、jiro.name, jiro.height, jiro.weightがそれぞれtaro.name, taro.height, taro.weightに代入される。

上記の例のように、構造体のメンバーに配列がある場合、構造体を代入するとそのメンバーの配列はコピーされる。配列単独では代入はできないが。関数の引数が構造体の場合も同様で、実引数の構造体が仮引数に代入されるが、その際、構造体のメンバーに配列があればコピーされる。

配列のコピーについて（教科書p. 130）

int a[5]; int b[5];と宣言すると、aとbは型は同じだが、b=aやa=bという代入式は書けない(a, bはそれぞれ&a[0], &b[0]に変換され、アドレスを持たない式なので)。以下のように要素毎に代入を行う必要がある。

```
for (i = 0; i < 5; i = i+1)
    b[i] = a[i];
```

復習

配列型の式eの値は、(sizeofの引数、&の引数の場合を除いて)その先頭要素e[0]のアドレスである。(この場合、式eは式&e[0]で置き換えても同じ。)

関数への構造体データの受け渡し(構造体をコピーする例) (打ち込んで確認)

```
#include <stdio.h>
typedef struct {
    char name[20]; /* 名前 */
    int height;    /* 身長 */
    double weight; /* 体重 */
} student;
void print_data( student std )
{
    printf("%sの身長は%dcm、体重は%fkgです。¥n",
           std.name, std.height, std.weight);
}
int main(void)
{
    student taro = {"Taro", 176, 64.5};
    print_data( taro );
    return 0;
}
```

構造体taroのコピーがstdに代入されてから、関数print_dataの本体が実行される。

関数への構造体データの受け渡し(ポインタを渡す例) (打ち込んで確認)

```
#include <stdio.h>
typedef struct {
    char  name[20]; /* 名前 */
    int   height;    /* 身長 */
    double weight;   /* 体重 */
} student;
void change_data( student * std )
{
    (*std).height = 180;
    (*std).weight = 80.0;
}
int main(void)
{
    student taro = {"Taro", 176, 64.5};
    change_data( &taro );
    printf("%sの身長は%dcｍ、体重は%fkｇです。¥n",
           taro.name, taro.height, taro.weight);
    return 0;
}
```

構造体taroへのポインタを受け取る。

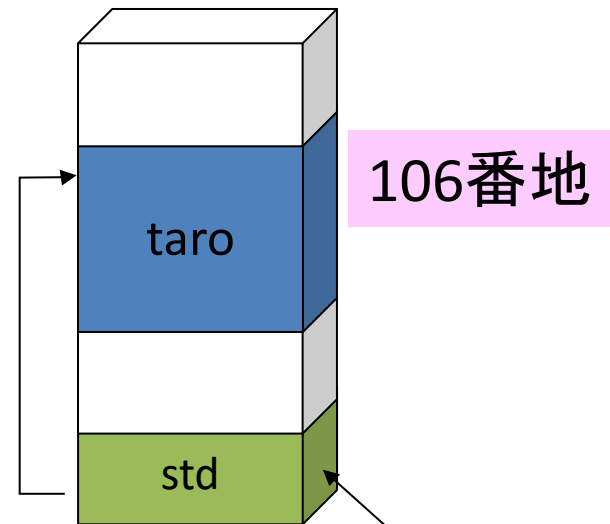
構造体のポインタを関数へ渡す場合 の解説

```
void change_data( student * std )  
{  
    (*std).height = 180;  
    (*std).weight = 80.0;  
}
```

change_data(&taro);

(注意) *std.heightと書くと、
*(std.height)と解釈される。
(ドット演算子の方が*より優先順位が高いため)

*stdはtaro の別名(alias)
になる。



student型のオブ
ジェクトのアドレス
を入れるための箱

アロー演算子 ->

```
void change_data (student *std ) {  
    (*std).height = 180;  
    (*std).weight = 80.0;  
}
```

=

```
void change_data (student * std ) {  
    std->height = 180;  
    std->weight = 80.0;  
}
```

式 e が、構造体 (τ 型) へのポインタ型 ($\tau * \text{型}$) の式の時、その構造体のメンバ m は

$(*e).m$

で得られるが、これは

$e \rightarrow m$

と書いてもよい (syntax sugar)。

例(打ち込んで確認)

```
#include <stdio.h>
typedef struct {
    char name[20]; /* 名前 */
    int height;     /* 身長 */
    double weight;  /* 体重 */
} student;
void change_data(student * std )
{
    std->height = 180;
    std->weight = 80.0;
}
int main(void)
{
    student taro = {"Taro", 176, 64.5};
    change_data( &taro );
    printf("%sの身長は%dcm、体重は%fkgです。¥n",
           taro.name, taro.height, taro.weight);
    return 0;
}
```

構造体を返す関数(打ち込んで確認)

```
#include <stdio.h>
typedef struct{
    int x;
    int y;
} point;
point makePoint (int x, int y) {
    point p;
    p.x = x;
    p.y = y;
    return p;
}
int main (void) {
    point p;
    p = makePoint (2,3);
    printf ("(x,y) = (%d, %d)¥n", p.x, p.y);
    return 0;
}
```

xy平面上の点を表すために、int型の変数x,yからなる構造体型を定義し、それにpointという名前をつけている。

(注意) 構造体内の配列について

- 構造体内に配列があるとき、
 - 構造体を関数に渡したら、構造体内の配列の各要素はコピーされる。(よって、関数内で配列の値を書き変えても呼び出し元の配列には影響がない。)
 - 構造体を構造体型の変数に代入したら、構造体内の配列の各要素はコピーされる。(よって、片方の構造体内の配列の値を書き変えてももう片方の構造体の配列の値には影響がない。)

基本課題1

キーボードから3人分の学生の学籍番号、名前、英語の点数を入力したのち、学籍番号を入力することにより、その番号の学生の情報が右の実行例のように表示されるようにせよ。ただし、名前はアルファベットの文字列(char型の配列)で空白を含まないものとし、学籍番号と英語の点数はint型の数とする。また、一人の学生の情報を表すために以下の構造体型を用いよ。

```
struct {  
    int id;  
    char name[20];  
    int english;  
}
```

[実行例]

1人目

学籍番号: 10001

名前: Taro

英語: 90

2人目

学籍番号: 10002

名前: Jiro

英語: 70

3人目

学籍番号: 10003

名前: Saburo

英語: 60

登録完了

探したい人の学籍番号を入力: 10002

学籍番号: 10002, 名前: Jiro, 英語: 70

基本課題2

キーボードから2つの複素数を読み込み、その2つの複素数の和を下の実行例のように出力するプログラムを作成せよ。課題を簡単にするため、入力する複素数は実数部、虚数部ともに正のint型の数であると仮定してよい。ただし、複素数を表すために、以下の構造体 `complex` を用い、複素数の和は、関数を使って求めよ。

```
typedef struct {  
    int re;  
    int im;  
} complex;
```

和を求める関数は、複素数を表す `complex` 型の引数 `c1`, `c2` を受け取って、それらの和を表す `complex` 型の値を返す関数として定義せよ。

```
complex sum (complex c1, complex c2) {    ....    }
```

[実行例]

複素数aの実数部を入力してください: 2

複素数aの虚数部を入力してください: 3

複素数bの実数部を入力してください: 4

複素数bの虚数部を入力してください: 5

$a + b = 6 + 8i$ です。

発展課題1

キーボードから3人分の学生の名前、身長、体重を入力したのち、名前を入力することにより、その学生のBMIが画面上に表示されるようにせよ。ただし、名前はアルファベットの文字列(char型の配列)で空白を含まないものとし、身長はint型(単位はcm)、体重はdouble型(単位はkg)とする。また、一人の学生の情報を表すために以下の構造体型studentを用いよ。また、student型を引数にとり、BMIを画面上に表示する関数showBMIを

```
void showBMI (student s) { ... }
```

の形で定義し、それをmain関数中から呼び出す形でプログラムを作成せよ。BMIは小数点以下一桁まで表示せよ。(printfの変換指定子 %.1f)

```
typedef struct {  
    char name[20];  
    int height;  
    double weight;  
} student;
```

[BMIの計算式]

体重(kg) / (身長(m)の2乗)

(ヒント) 文字列比較関数strcmpを用いてよい。関数strcmpの使い方はman strcmpで調べよ。

[実行例]

1人目

名前: Taro

身長: 176

体重: 64.5

2人目

名前: Jiro

身長: 165

体重: 55.5

3人目

名前: Saburo

身長: 168

体重: 70.0

登録完了

探したい人の名前を入力: Jiro

JiroのBMIは20.4です。

発展課題2

基本課題2と同様のことを、複素数の積について行え。積を求める関数は、複素数を表すcomplex型の引数c1, c2を受け取って、それらの積を表すcomplex型の値を返す関数として定義せよ。

```
complex prod (complex c1, complex c2) {  
    ....  
}
```

ただし、実数部、虚数部に条件はつけない。結果の虚数部が負になった場合は、“ $2 + -3i$ ”のように表示するのではなく、“ $2 - 3i$ ”のように表示せよ。また、実数部と虚数部の片方が0になった場合は、0でない方のみを表示するようにせよ。実数部と虚数部の両方が0になった場合は、0と表示せよ。

[実行例]

複素数aの実数部を入力してください: 1

複素数aの虚数部を入力してください: 1

複素数bの実数部を入力してください: -1

複素数bの虚数部を入力してください: -1

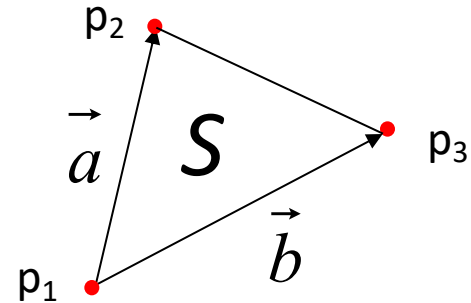
$a * b = -2i$ です。

発展課題3

キーボードから3点の2次元座標値(x, y)をdouble型で読み込み, 3点から作られる3角形の面積Sを次ページの実行例のようにdouble型で出力するプログラムを作成せよ。

$$\vec{a} = (a_x, a_y) \quad \vec{b} = (b_x, b_y)$$

$$\text{のとき、} S = \frac{1}{2} |a_x b_y - a_y b_x|$$



但し, 座標を格納する構造体として以下のものを用いること。

```
typedef struct {  
    double x;  
    double y;  
} point;
```

(注) double型の値の読み込みは、
scanf ("%lf", &p.x);
のようにする(pがpoint型の場合)。printfでの表示は、
printf ("%f", p.x);
のようにする。

面積を求める関数は、座標を表すpoint型の引数p1, p2, p3を受け取り、面積をdouble型で求め、それを返り値として返す関数として定義せよ。

```
double area (point p1, point p2, point p3) { ... }
```

面積は小数点以下一桁まで表示せよ。(printfの変換指定子 %.1f)

発展課題3 実行例

[実行例]

3点p1, p2, p3の座標を入力してください:

p1のx座標: 1.0

p1のy座標: 1.1

p2のx座標: 3.1

p2のy座標: 1.2

p3のx座標: 2.2

p3のy座標: 4.4

p1, p2, p3で作られる3角形の面積は3.4です。

発展課題4

キーボードから3人分の学生の名前、身長を入力したのち、基準となる身長を入力し、身長がそれ以上の学生の名前をコンマで区切って入力された順番で以下の実行例のように表示するプログラムを作成せよ。ただし、名前はアルファベットの文字列(char型の配列)で空白を含まないものとし、身長はint型(単位はcm)とする。また、一人の学生の情報を表すために以下の構造体型 student を用いよ。

```
typedef struct {  
    char name[20];  
    int height;  
} student;
```

[実行例]

1人目

名前: Taro

身長: 176

2人目

名前: Jiro

身長: 165

3人目

名前: Saburo

身長: 168

登録完了

基準となる身長(cm)を入力: 167

身長が167cm以上の学生: Taro, Saburo

参考課題1

キーボードから3人分の名前および数学、英語の点数をint型で入力し、各科目の平均点をdouble型で求め、小数点以下1桁まで表示せよ。(printfの変換指定子 %.1f)

[実行例]

1人目

名前: Taro

数学: 80

英語: 90

2人目

名前: Jiro

数学: 70

英語: 70

3人目

名前: Saburo

数学: 90

英語: 60

数学の平均点は80.0点, 英語の平均点は73.3点です。

参考課題の解答例

```
#include<stdio.h>
typedef struct{
    char name[20];
    int math;
    int english;
}student;

int main(void){
    student s[3];
    int a, i, sum;
    double mAver, eAver;
    for(i=0;i<3;i++){
        printf("%d人目¥n",i+1);
        printf("名前: ");
        scanf("%s",s[i].name);
        printf("数学: ");
        scanf("%d",&s[i].math);
        printf("英語: ");
        scanf("%d",&s[i].english);
    }
```

```
/* 続き */
sum=0;
for(i=0;i<3;i++)
    sum=sum+s[i].math;
mAver = (double) sum / 3;
sum=0;
for(i=0;i<3;i++)
    sum=sum+s[i].english;
eAver = (double) sum / 3;
printf("数学の平均点は%.1f点, 英語の  
平均点は%.1f点です。¥n",
        mAver, eAver);
return 0;
}
```

補足

- 同じ構造体型 (struct {...}) の記述が同じファイル (翻訳単位) にあった場合、それらは違う型と解釈される。同じファイル内の2箇所以上で同じ構造体型 (struct {...}) で変数を宣言した場合、それらの間で代入をすることはできない。また、関数の仮引数宣言部分に同じ構造体型 (struct {...}) の記述をしてあっても引数として渡すことはできない。(ISO C11 6.7.2.3節の項目5を参照)
- ただし、別のファイル (翻訳単位) に同じ構造体型の記述があった場合、それらは同じ型とみなされる。(これを許さないと分割コンパイルをするときに困る。)(ISO C11 6.2.7節の項目1を参照)